

Reporte de práctica: Comunicación I2C entre 4 Arduinos

Alumno	Jesús Antonio Rodríguez Peraza
Matrícula	L231000105
Materia	Sistemas Programables
Práctica	6 — Comunicación I2C entre 4 Arduinos
Plataforma	Tinkercad (simulación) — 4× Arduino UNO R4 WiFi

Objetivo

Comprender el funcionamiento del bus I2C mediante la comunicación entre un Arduino maestro y tres Arduinos esclavos, que comparten las líneas SDA y SCL y se identifican con direcciones distintas (0x08, 0x09 y 0x0A). El alumno debe ser capaz de armar el bus, asignar direcciones y programar el envío y la recepción de datos.

Metodología

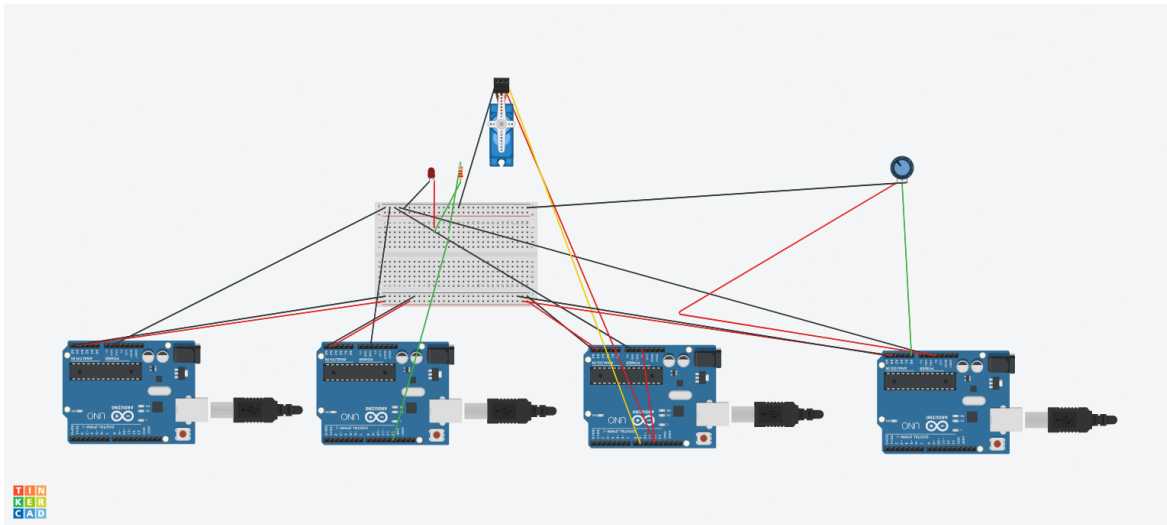
Se armó en Tinkercad un bus I2C con cuatro Arduino UNO R4 WiFi, todos compartiendo las líneas SDA (A4), SCL (A5) y tierra (GND). Cada esclavo corre su propio programa y responde solo cuando el maestro lo llama por su dirección:

- **Esclavo 1 (0x08):** recibe una orden del maestro y enciende o apaga un LED.
- **Esclavo 2 (0x09):** recibe un ángulo (0° – 180°) del maestro y mueve un servomotor.
- **Esclavo 3 (0x0A):** lee un potenciómetro (A0) y envía su valor de 10 bits al maestro cuando este lo pide, en dos bytes.

El maestro corre en `loop()` sin bloquear: cada 500 ms (con `millis()`, no de `delay()`) pide el valor del potenciómetro al esclavo 3, lo convierte a un ángulo de 0° a 180° con `map()` y se lo manda al esclavo 2. Al mismo tiempo revisa el Monitor Serie: si el usuario escribe 1 o 0, le manda esa orden al esclavo 1 para prender o apagar el LED. Cada envío revisa el resultado de `Wire.endTransmission()` (o de `Wire.requestFrom()`) para avisar en el Monitor Serie si algún esclavo no respondió, en vez de que el programa se quede esperando.

Al usar la UNO R4 WiFi en vez de la UNO R3 original de la consigna, se hicieron cuatro ajustes: (1) el maestro espera a que se abra el puerto serie USB nativo antes de imprimir (`while (!Serial && millis() < 3000)`), porque en la R3 los primeros mensajes se perdían; (2) la resistencia del LED sube de $220\ \Omega$ a $470\ \Omega$, porque los pines de la R4 soportan menos corriente; (3) las resistencias de pull-up de $4.7\ k\Omega$ en SDA y SCL dejan de ser opcionales; y (4) se evitó nombrar una constante `PIN_LED` en el esclavo 1, porque ese nombre ya lo usa el core de la R4 para el LED integrado y el programa no compilaba (error verificado con `arduino-cli` antes de corregirlo).

Diagrama de conexión



Captura del circuito armado en Tinkercad: 4× Arduino UNO, protoboard con LED + resistencia y potenciómetro, micro servo, bus I2C (SDA/SCL) y tierra común entre las cuatro placas.

Resultados

Comportamiento esperado del diseño, a confirmar corriendo la simulación completa en Tinkercad (ver nota de pendientes en Resultados/ y Terminal/): el maestro debe imprimir en el Monitor Serie una línea cada 500 ms con el valor crudo del potenciómetro (0–1023) y el ángulo equivalente (0°–180°) enviado al esclavo 2, cuyo servo debe seguir ese ángulo en tiempo real. Al escribir 1 o 0 en el Monitor Serie del maestro, el LED del esclavo 1 debe prender o apagar de inmediato. Si se desconecta cualquier esclavo del bus, el maestro debe reportarlo por su nombre en vez de congelarse.

PENDIENTE: capturas reales del Monitor Serie y video de la simulación corriendo (ver Terminal/Readme.txt y Video/Readme.txt).

Conclusiones

El bus I2C permite comunicar varios dispositivos usando solo dos líneas (SDA y SCL) más tierra común; agregar un esclavo no pide pines nuevos, solo una dirección distinta. El maestro controla la comunicación (decide con quién habla, cuándo, y si pide o manda datos), mientras que cada esclavo solo responde cuando se le llama por su dirección — si dos comparten la misma, ambos contestan a la vez y el dato se corrompe.

Revisar el resultado de `endTransmission()` y de `requestFrom()` es lo que distingue un fallo detectado (NACK reportado en el Monitor Serie) de un programa que se cuelga esperando una respuesta que no va a llegar. Usar `millis()` en vez de `delay()` en el maestro fue necesario por la misma razón que en la Práctica 2: el maestro tiene que seguir atendiendo el Monitor Serie mientras espera su turno de consultar al esclavo 3.

Dos errores de armado quedaron documentados como los más comunes en esta práctica: una resistencia con ambas patas en la misma columna de la protoboard no protege nada (los huecos de una columna están unidos por dentro) y puede quemar el LED; y leer un dato de I2C más de una vez dentro de la misma expresión devuelve basura en la segunda lectura, porque el byte ya se consumió — hay que guardarlo en una variable antes de reutilizarlo.